

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



Deep Learning and Its Applications (CO3133)

Semester 261 – Class CC01 – Group G-02

Assignment 1 – Milestone 1

M1 Draft

Linear and MLP on Fashion-MNIST

Lecturer: Lê Thành Sách

Students: Võ Thị Như Quỳnh – 2353041
Mai Trung Kiên – 2352641
Hứa Tuệ Minh – 2450018

HO CHI MINH CITY, OCTOBER 2026



TABLE OF CONTENTS

1	Problem and data description	2
1.1	Problem statement	2
1.2	Dataset source and scope	2
1.3	Read the dataset	2
1.4	Define the train and validation subsets	3
1.5	Look at one image	3
1.6	Class distribution and imbalance	4
1.7	Representative training images	5
1.8	Pixel intensity distribution	6
1.9	Brightness, contrast and foreground coverage	7
1.10	Average appearance of each class	10
1.11	PCA: how much variation fits in fewer dimensions?	12
1.12	Two-dimensional views of the same images	14
1.13	Inspect preprocessing	15
1.14	EDA conclusions	16
2	Methodology	16
2.1	Pipeline	16
2.2	Data and inputs	17
2.3	Model definitions	17
2.4	Training and selection	18
2.5	Loss, metrics and timing	18
2.6	Recorded environment	19
2.7	Experiment design	19
2.8	Training and validation loop	19
2.9	Reproduction	20
3	Implementation results	20
3.1	Validation comparison	20
3.2	Held-out test results	21
3.3	Learning curves	21
3.4	Test confusion matrices	23
3.5	Inspect predictions	26
3.6	Discussion	27



1 Problem and data description

1.1 Problem statement

Predict one of ten clothing classes from a 28×28 grayscale image for automatic product categorization. Similar upper-body silhouettes and limited detail make some classes difficult to separate.

Given $x \in \mathbb{R}^{1 \times 28 \times 28}$, learn ten class scores $f_\theta(x)$ and predict $\hat{y} = \operatorname{argmax} f_\theta(x)$. M1 compares a linear classifier and an MLP with one hidden layer, measuring accuracy against parameter and runtime costs.

1.2 Dataset source and scope

We use the 2017 [Fashion-MNIST release by Zalando Research](#), described by Xiao, Rasul and Vollgraf and distributed under the [MIT license](#). It is our sole dataset; each product image has one supplied category label.

Fashion-MNIST contains 60,000 official training images and 10,000 test images, stored as IDX arrays with pixel values from 0 to 255. The classes are T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag and Ankle boot.

1.3 Read the dataset

Read the official image and label arrays. IDX image headers occupy 16 bytes; label headers occupy 8 bytes.

Set	Image shape	Label shape	Pixel dtype	Min	Max
Original train	(60000, 28, 28)	(60000,)	uint8	0	255
Official test	(10000, 28, 28)	(10000,)	uint8	0	255

There are 60,000 original training images and 10,000 official test images. Each image is 28×28 with one grayscale channel, stored as unsigned integers from 0 to 255. Image and label counts match in both sets; the image arrays have no explicit channel dimension.

File	Bytes	MiB
train-images-idx3-ubyte	47040016	44.861
train-labels-idx1-ubyte	60008	0.057
t10k-images-idx3-ubyte	7840016	7.477
t10k-labels-idx1-ubyte	10008	0.010
Total	54950048	52.404

The four uncompressed IDX files occupy 54,950,048 bytes (52.404 MiB) on disk, including their headers. Images store one unsigned byte per pixel; labels store one unsigned byte per image. The downloaded .gz files are compressed copies, while the arrays above read the uncompressed files directly.

1.4 Define the train and validation subsets

Reserve 10% of the original training set for validation. Use only the remaining training images for detailed EDA and fitted transformations.

Class	Train	Validation	Official test
T-shirt/top	5400	600	1000
Trouser	5400	600	1000
Pullover	5400	600	1000
Dress	5400	600	1000
Coat	5400	600	1000
Sandal	5400	600	1000
Shirt	5400	600	1000
Sneaker	5400	600	1000
Bag	5400	600	1000
Ankle boot	5400	600	1000
Total	54000	6000	10000

The split contains 54,000 training and 6,000 validation images. Every class contributes 5,400 training and 600 validation examples, so stratification preserves balance. The 10,000 official test images remain separate. Later image galleries and projection samples come from the training subset.

1.5 Look at one image

Display an actual training image at its native resolution. The ID refers to its row in the original training file.

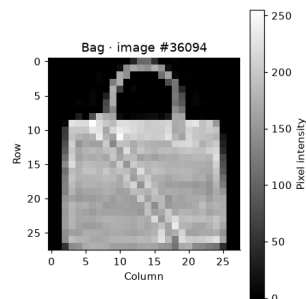


Figure 1: One training example at native 28×28 resolution; the ID indexes the original training array.

The image is made of discrete grayscale pixels: dark values form the background and lighter values describe the item. At this resolution, fine texture and small details are limited. The displayed label identifies a clothing category rather than a particular product.

1.5.1 Pixel values and the flattened input

Zoom into an 8×8 patch and show the same image as a vector.

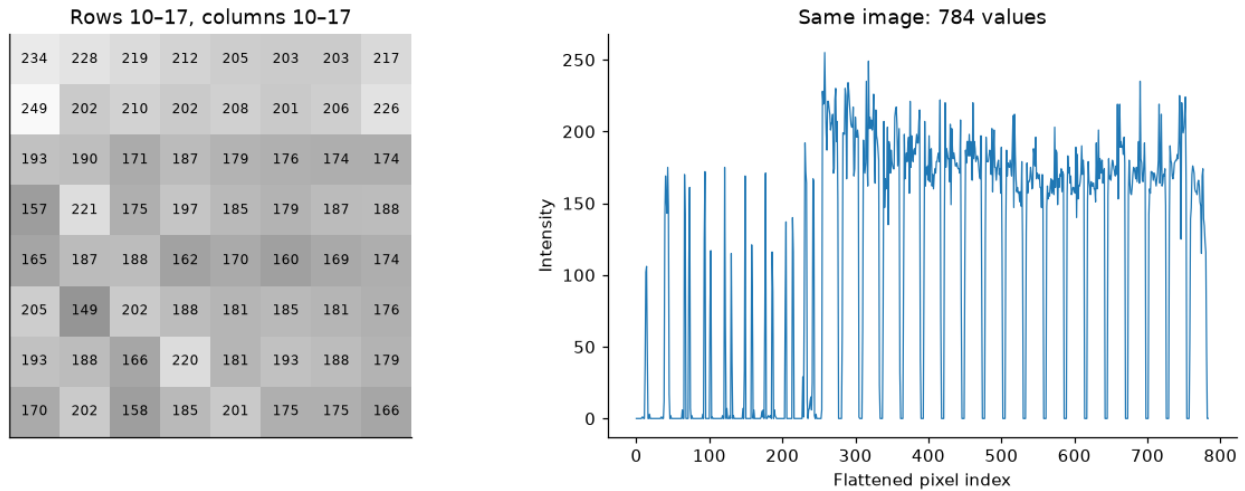


Figure 2: An 8×8 image patch and the same image flattened to 784 pixel values.

The patch includes intermediate gray values, so the image is not a binary silhouette. Flattening gives 784 features and preserves every pixel value in row-major order. A flat vector does not explicitly encode which pixels were adjacent in two dimensions; no resizing is needed.

1.6 Class distribution and imbalance

Count each class in the two original dataset partitions.

Class	Original train	Official test	Train share (%)	Test share (%)
T-shirt/top	6000	1000	10.0	10.0
Trouser	6000	1000	10.0	10.0
Pullover	6000	1000	10.0	10.0
Dress	6000	1000	10.0	10.0
Coat	6000	1000	10.0	10.0
Sandal	6000	1000	10.0	10.0
Shirt	6000	1000	10.0	10.0
Sneaker	6000	1000	10.0	10.0
Bag	6000	1000	10.0	10.0
Ankle boot	6000	1000	10.0	10.0

Every class has 6,000 original training examples and 1,000 official test examples: 10% of each partition. The largest-to-smallest class-count ratio is 1.0 in both sets. There is no class-frequency imbalance to correct, although some categories can still be harder to distinguish visually.

1.6.1 Visual comparison of class counts

Show the original class counts side by side.

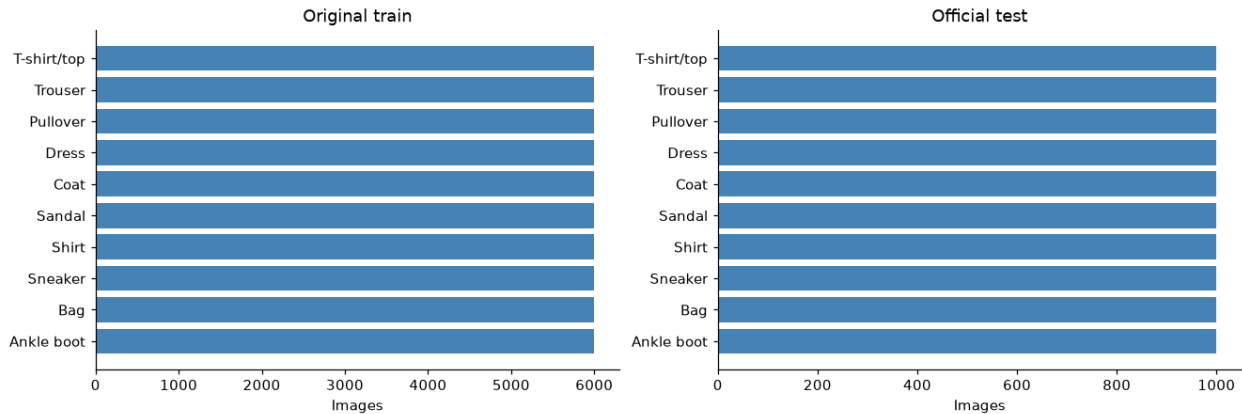


Figure 3: Official training and test class counts. Each partition has ten equally frequent classes.

The bars have equal length within each partition. The separate axes show different absolute counts, so compare class proportions rather than bar lengths across panels. An unweighted classification loss is a reasonable baseline; per-class metrics will still be useful.

1.7 Representative training images

Inspect four seeded examples from each class. First, look at T-shirts, trousers, pullovers, dresses and coats.



Figure 4: Four seeded training examples per class: T-shirt/top, Trouser, Pullover, Dress and Coat.

Examples within one category differ in brightness, outline and detail. Trousers have a distinctive two-leg silhouette. Pullovers and coats share a torso-and-sleeves structure, while dresses vary in length and shape. Four examples illustrate variation but cannot establish how common each appearance is.

1.7.1 Footwear, shirts and bags

Inspect the remaining five classes using the same sampling rule.



Figure 5: Four seeded training examples per class: Sandal, Shirt, Sneaker, Bag and Ankle boot.

Footwear often has a horizontal silhouette, but sandals, sneakers and ankle boots differ in how much of the foot or ankle they cover. Bags vary in body shape and handle visibility. Shirts share visual elements with several upper-body categories, making their class boundaries worth inspecting more closely.

1.8 Pixel intensity distribution

Count intensities across all 54,000 training images. Use a logarithmic vertical axis so the background spike does not hide the other intensities.

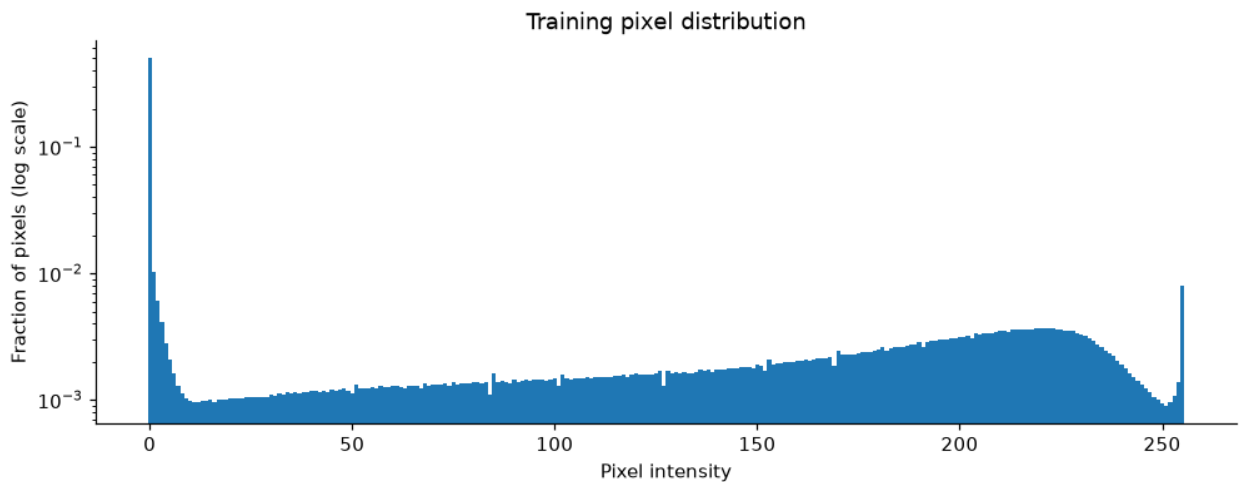


Figure 6: Training pixel-intensity distribution on a logarithmic vertical scale.

Statistic	Value
Zero-valued pixels (%)	50.2143
Completely black images	0.0000

About half of the training pixels are exactly zero, reflecting the large dark background regions. Other pixels span many intensities, so foreground texture is retained. A high proportion of zero pixels does not mean the corresponding images are empty; the all-black count distinguishes these cases.

1.8.1 Where do pixels vary?

Compare the mean image with variance at each pixel location. Both summaries use intensities scaled to [0, 1].

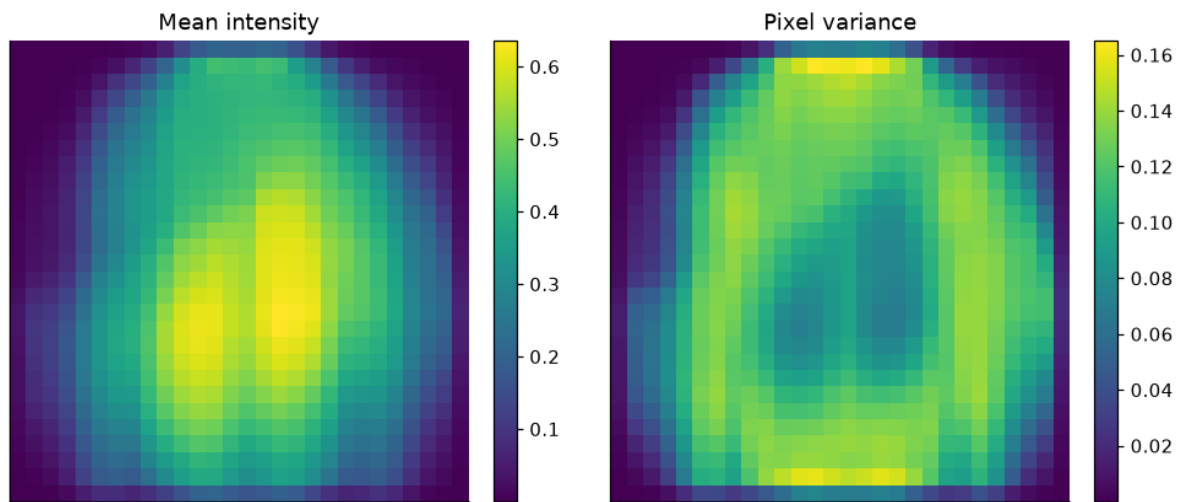


Figure 7: Mean and variance at each pixel coordinate across the 54,000 training images.

The mean image shows where clothing tends to occupy the frame, while variance highlights locations that change across examples. Background corners are comparatively stable. Low variance alone does not establish that a coordinate is useless for distinguishing classes, so these plots do not justify removing pixels.

1.9 Brightness, contrast and foreground coverage

Summarize each image. Brightness is mean intensity, contrast is intensity standard deviation, and foreground means a raw pixel value above 26.

Statistic	Brightness	Contrast	Coverage
count	54000.000	54000.000	54000.000
mean	0.286	0.320	0.451
std	0.126	0.078	0.149
min	0.019	0.070	0.038
5%	0.101	0.178	0.214
50%	0.272	0.332	0.458
95%	0.503	0.428	0.673
max	0.752	0.475	0.909

These features describe global appearance, not image correctness. Brightness combines item intensity and occupied area; coverage depends on the chosen threshold. Extreme values should lead to visual inspection before any decision to remove or alter an image.

1.9.1 Compare appearance by class

Show the within-class distributions. Outlier markers are hidden for readability; all images remain included in the boxplot calculations.

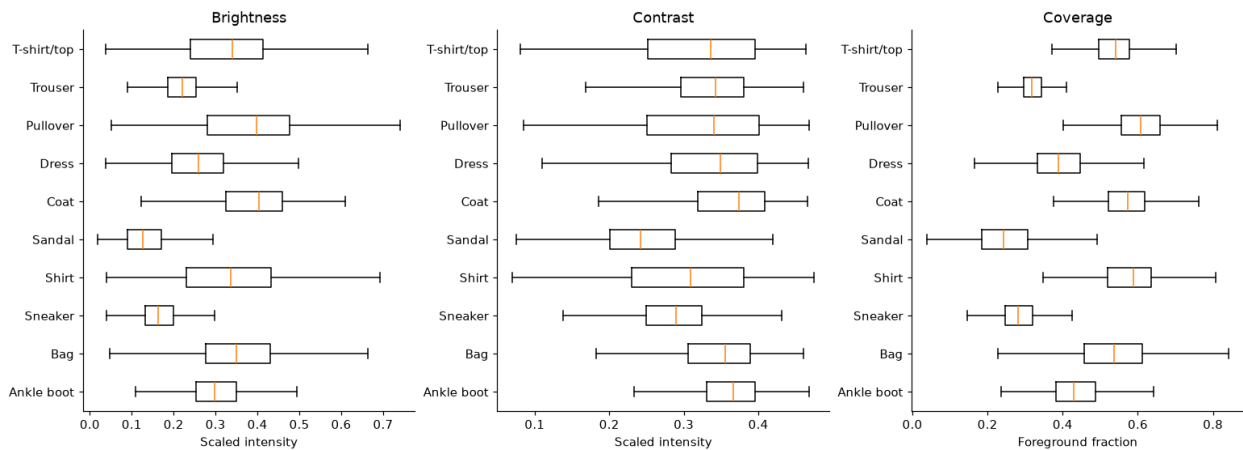


Figure 8: Brightness, contrast and threshold-based foreground coverage by class.

The class distributions overlap, so none of these single-number features cleanly identifies every category. Sandals occupy relatively little foreground area, whereas fuller garments cover more of the frame. Differences in median brightness can reflect silhouette and background as well as fabric appearance.

1.9.2 Inspect the extreme examples

For each feature, display the two lowest and two highest training examples.

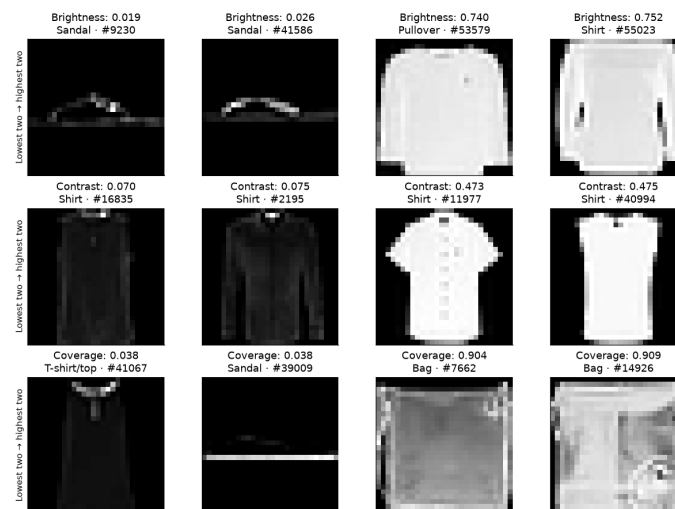


Figure 9: Deliberately selected extremes: two lowest and two highest examples for each appearance statistic.

The lowest-brightness examples are dark sandals; the lowest-contrast examples are dark shirts. The two highest-coverage examples are bags that nearly fill the frame. Sandal #39009 has an unusual large bright region below the footwear, which deserves closer inspection before proposing a cleaning rule. These deliberately selected extremes are not representative examples or confirmed labeling errors.

1.9.3 What does the foreground threshold select?

Compare an image with its threshold mask.

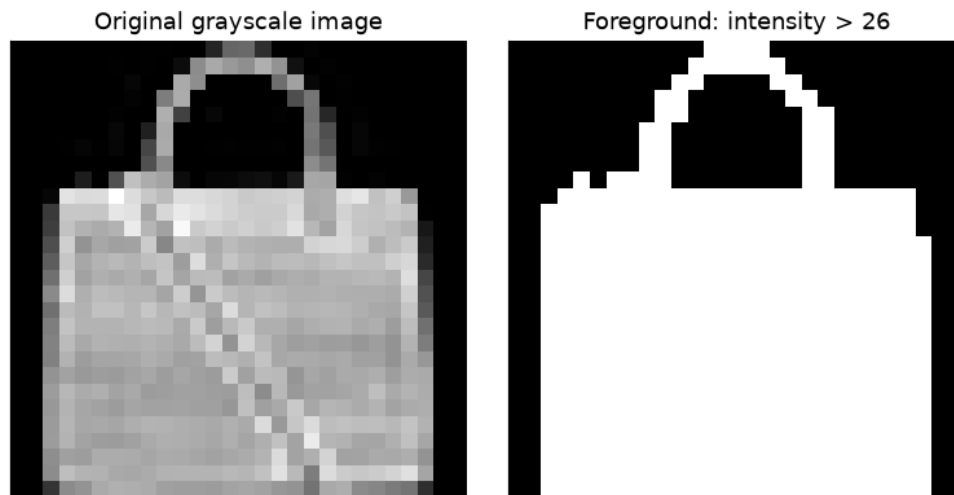


Figure 10: Raw grayscale input versus a threshold-based foreground mask used only for exploration.

The mask simplifies the grayscale image into occupied and unoccupied pixels. It can exclude dark parts of the garment, so coverage is a threshold-based proxy rather than a precise segmentation. The original grayscale image remains the input representation.

1.9.4 Sensitivity to the foreground threshold

Change the threshold by ten intensity levels in either direction and compare median coverage by class.

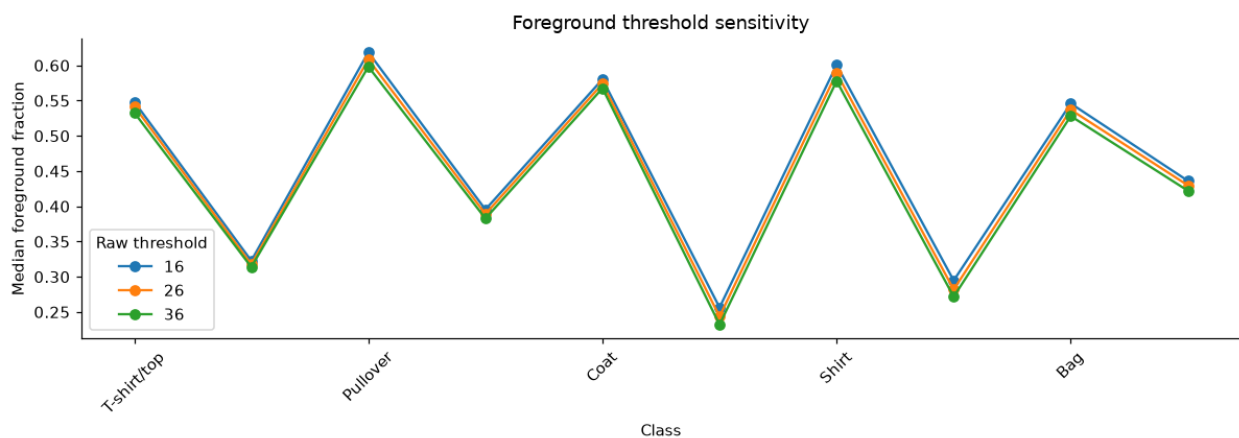


Figure 11: Sensitivity of median foreground coverage to raw thresholds 16, 26 and 36.

Increasing the threshold reduces measured foreground coverage. Compare the spacing and ordering of the curves before treating a class difference as robust. The thresholds describe the same unmodified images; they do not define alternative cleaned datasets.

1.10 Average appearance of each class

Average the training images within each class, using a shared grayscale range.

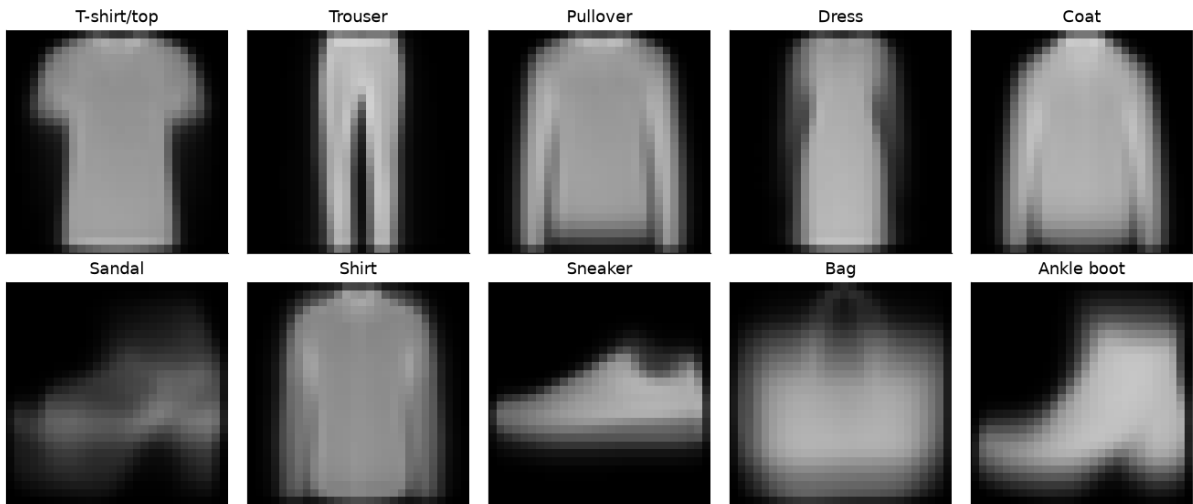


Figure 12: Mean training image for each class, displayed on a common intensity scale.

Averaging preserves common silhouettes while blurring details and variations in pose. Trousers and footwear retain characteristic shapes; several upper-body categories have similar outlines. These are synthetic mean images, not actual products or representative individual examples.

1.10.1 Compare class means

Measure root-mean-square (RMS) distance between class means on the [0, 1] intensity scale.

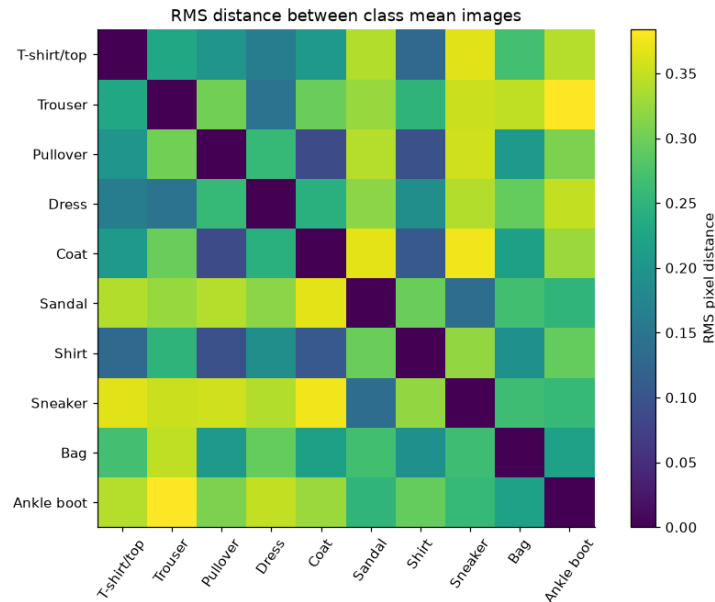


Figure 13: RMS distance between class mean images; lower values indicate similar mean appearances.

Dark off-diagonal cells indicate similar average appearances. This comparison suggests class pairs to inspect visually. It is not a confusion matrix: similarity between means does not measure prediction errors.

1.10.2 Which average silhouettes are closest?

List the five smallest distances between distinct classes.

Class a	Class b	RMS distance
Pullover	Coat	0.088
Pullover	Shirt	0.096
Coat	Shirt	0.107
T-shirt/top	Shirt	0.132
Sandal	Sneaker	0.137

Pullover and Coat have the closest mean images. Their similar torso-and-sleeve structure is consistent with the earlier galleries. This is a hypothesis about visual ambiguity; class means can hide substantial differences among individual images.

1.10.3 Return to real images of the closest pair

Use the previously sampled examples to compare the closest two classes directly.



Figure 14: Individual Pullover and Coat examples, the two classes with the closest mean images.

The real examples show both shared silhouettes and differences that averaging smooths away. Necklines, openings, length and texture can matter, but they may be subtle at 28×28 resolution. Similarity in this small gallery does not imply that the categories are interchangeable.

1.11 PCA: how much variation fits in fewer dimensions?

Fit an exploratory projection on a balanced sample of 6,000 training images. PCA uses pixel variation, not labels; labels only define the balanced sample.

Components	Cumulative variance (%)
2	46.62
8	69.24
16	76.45
32	82.61
64	88.22
128	92.94

Two components retain about 46.62% of the variation in this 6,000-image training sample; 64 retain 88.22%, and 128 retain 92.94%. A two-dimensional view therefore discards more than half the measured variation. Retained variance is different from the category-specific information needed for classification.

1.11.1 Cumulative explained variance

View the tradeoff between the number of components and retained variation.

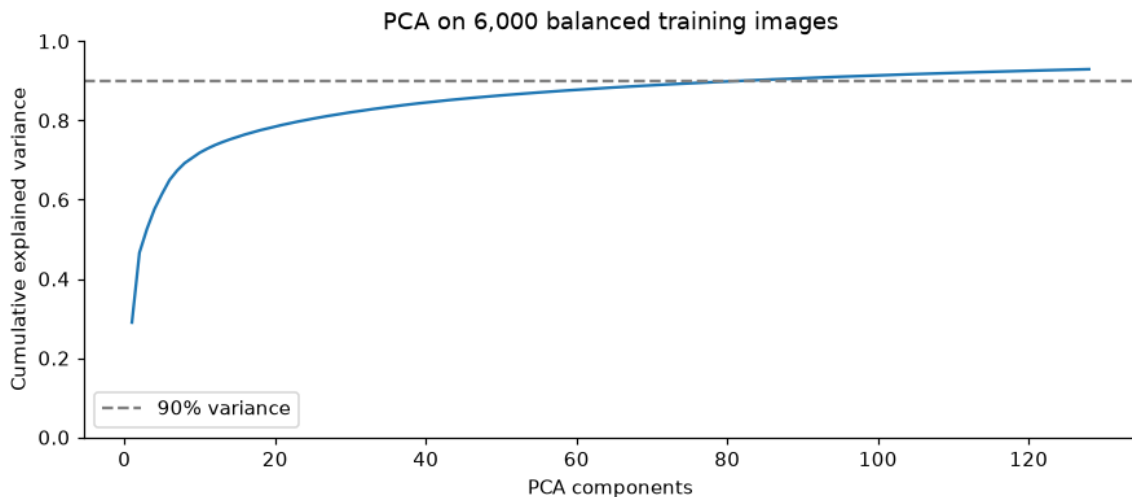


Figure 15: Cumulative explained variance from 128 PCA components fitted on 6,000 training images.

The curve rises quickly and then flattens: later components each add less variance. Only 128 components were computed, so targets above the endpoint cannot be assigned a component count from this run. A compression choice would need a separate evaluation of its effect on the eventual task.

1.11.2 Look at reconstructed images

Reconstruct one sampled image from each of four classes. Clip only the displayed intensities to $[0, 1]$.

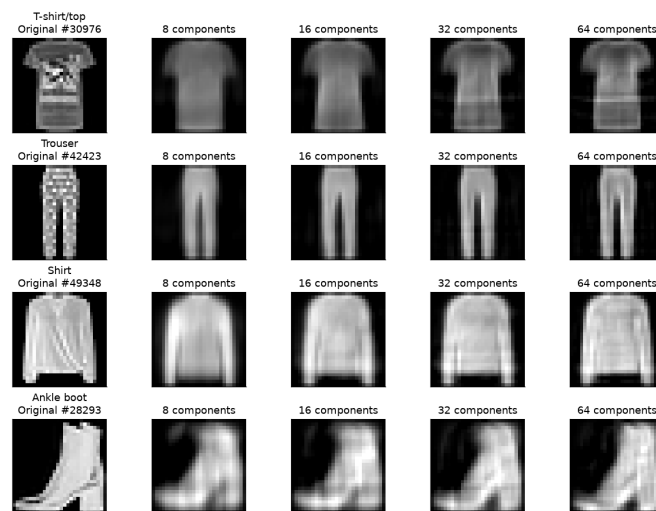


Figure 16: PCA reconstructions at four ranks. These examples are from the PCA fitting sample.

Broad silhouettes are recognizable with eight components, while more components sharpen sleeves, trouser legs and the boot outline. The patterned trousers and printed T-shirt still lose fine detail at 64 components. These images belong to the PCA fitting sample and illustrate reconstruction; they do not estimate performance on unseen data.

1.12 Two-dimensional views of the same images

Select 200 images per class from the PCA sample and plot their first two component scores.

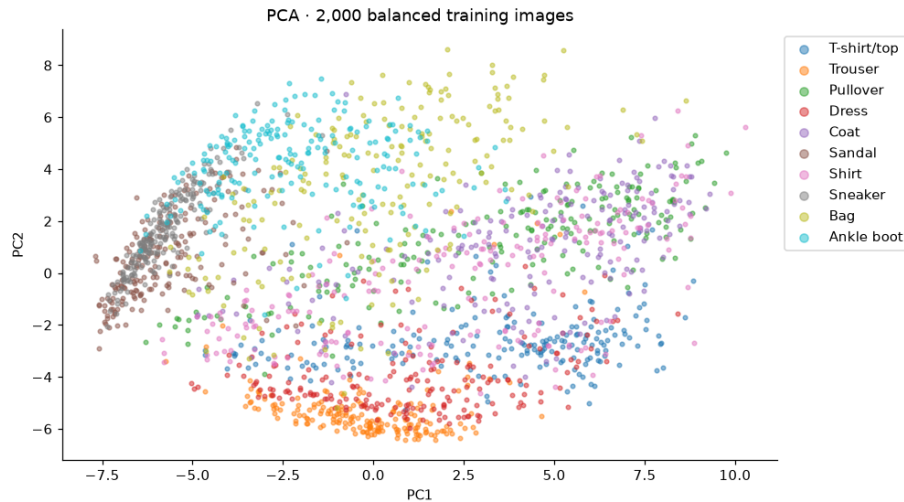


Figure 17: First two PCA components for a balanced 2,000-image training sample.

Several labels overlap in the first two components. PCA maximizes overall pixel variance, so its leading directions need not separate clothing classes. Overlap in this view does not show that the original 784-dimensional inputs are inseparable.

1.12.1 A nonlinear view with t-SNE

Embed the same 2,000 images using their first 50 PCA scores. This is an exploratory visualization, not classifier training.

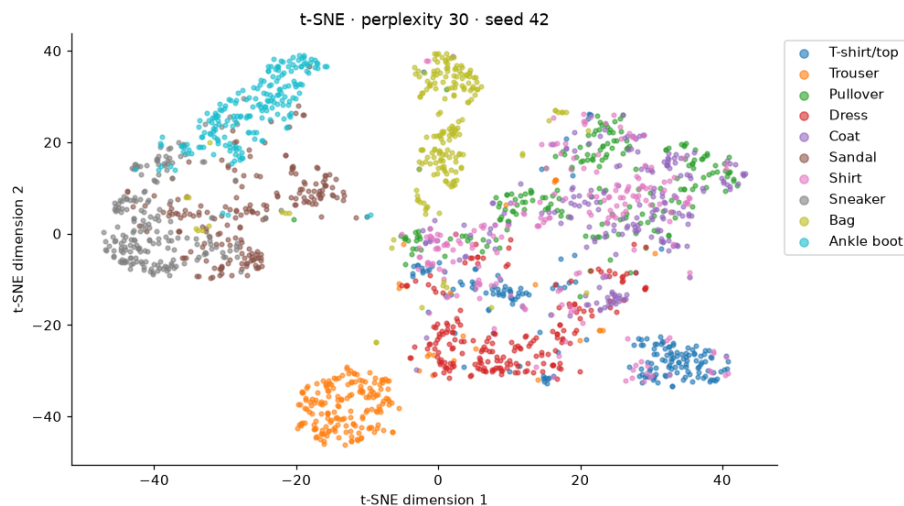


Figure 18: t-SNE of the same 2,000 training examples using the first 50 PCA scores.

Trousers form a compact neighborhood, while many pullover, coat and shirt points mix in the upper-body clothing region. Footwear occupies another part of the view, with some sandal and sneaker mixing. These local patterns support further image inspection. Distances between distant groups, apparent gaps and cluster sizes are not classification accuracy or reliable global similarity measures; the layout depends on sampling and settings.

1.13 Inspect preprocessing

Calculate normalization statistics from the 54,000 training images only. Scaling divides by 255; standardization then subtracts the training mean and divides by its standard deviation.

Representation	Mean	Population std	Min	Max
Raw	72.983297	90.055735	0.000000	255.000000
Scaled	0.286209	0.353160	0.000000	1.000000
Standardized	0.000000	1.000000	-0.810424	2.021156

The training-fitted mean and standard deviation on the $[0, 1]$ scale are approximately 0.286209 and 0.353160. Standardization gives zero mean and unit standard deviation over all training pixels together, not separately within each image. Validation and test inputs reuse these training statistics.

1.13.1 The same image before and after scaling

Use corresponding display ranges to compare the same image in three numerical representations.

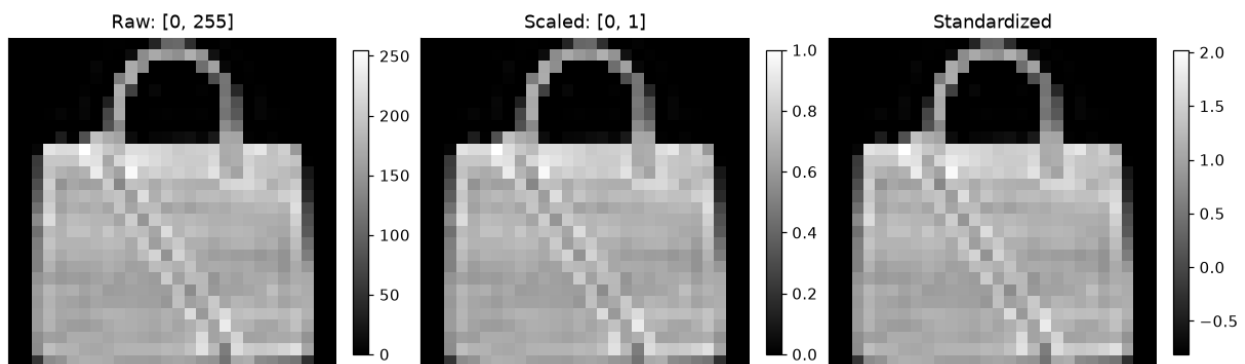


Figure 19: Identical image geometry under raw, scaled and standardized numerical representations.

The silhouettes remain identical because these transformations change numerical scale while preserving pixel order and relative contrast. The color bars show that the underlying values differ. Standardization includes negative values and is not restricted to $[0, 1]$.

1.14 EDA conclusions

Evidence from the exploration	Implication
Ten equally frequent classes in the original sets and stratified subsets	Start from an unweighted baseline; inspect class-specific performance later.
Fixed 28×28 grayscale images	Keep native resolution; a flattened image contains 784 values.
Large background regions and varied foreground intensities	Scaling is useful; zero pixels alone are not a reason to reject an image.
Broad brightness and coverage distributions within classes	Inspect unusual examples visually before proposing cleaning rules.
Pullover and Coat have similar mean silhouettes	Treat related upper-body categories as a hypothesis for later error analysis.
Two-dimensional projections discard information and mix some labels	Use them for exploration, not as an estimate of classifier performance.
Normalization statistics are calculated from training pixels only	Reuse the same mean and standard deviation for later validation and test inputs.

Limits. The galleries are small samples. Pixel distances and threshold masks describe appearance, not product identity or labeling correctness. PCA and t-SNE are sample-based exploratory views. This exploration provides no classification accuracy or evidence about which trained model is best.

Preprocessing decision. Separate training code can reproduce the split using seed 42, `test_size=0.10` and `stratify=y`, and compute normalization on that training subset. The EDA supports keeping all images and all pixel coordinates for an initial baseline.

2 Methodology

2.1 Pipeline

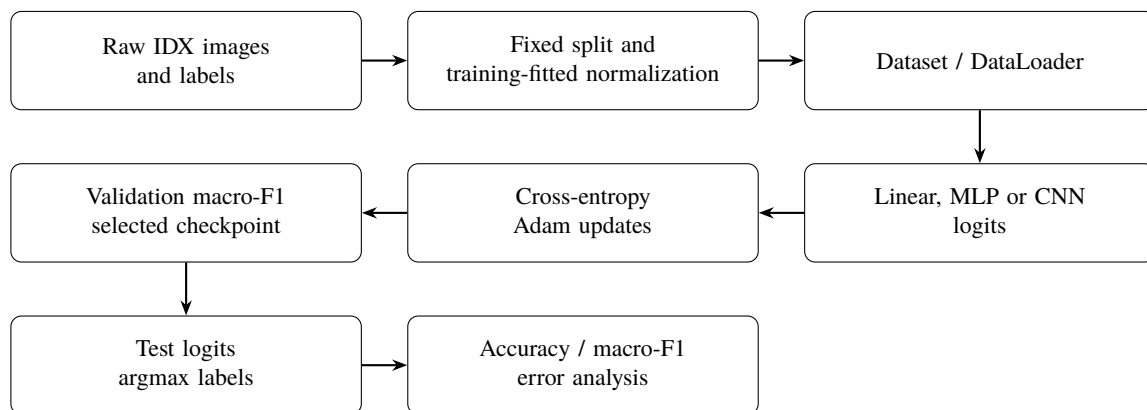


Figure 20: Shared training and evaluation pipeline.

The project implements data preparation, model definitions, training and evaluation. PyTorch provides layers, automatic differentiation and Adam; NumPy handles arrays, scikit-learn supplies splits and metrics, and Matplotlib produces the figures.

2.2 Data and inputs

A seed-42 stratified split divides 60,000 official training images into 54,000 training and 6,000 validation images; the 10,000-image test set is unchanged. Each class contributes 5,400, 600 and 1,000 images respectively. No images are removed or relabeled.

Checkpoints store disjoint training/validation indices. Test data neither selects weights nor fits normalization. Product identifiers are unavailable, so image-index separation cannot establish product-level independence.

```
1 x = (pixel / 255 - training_mean) / training_std
2 training_mean = 0.286209    training_std = 0.353160
3
4 images: float32 [B, 1, 28, 28]
5 labels: int64 [B]          values 0 ... 9
6 logits: float32 [B, 10]    output of every model
```

Training-only statistics normalize images once in the dataset constructor. Batch size is 128, with partial batches retained. Only training shuffles, using a seeded generator; loaders use zero workers. Inputs retain their native resolution and all pixel coordinates. FashionMNISTDataset implements `__len__` and `__getitem__` to supply normalized image-label pairs. An example full batch contains float32 images [128, 1, 28, 28] and int64 labels [128]; model logits have shape [128, 10].

2.3 Model definitions

B denotes batch size. Every model returns ten raw logits. Softmax is used for inspection probabilities, never before cross-entropy.

Model	Internal representation	Parameters
Linear	784 pixels $\rightarrow$ 10 logits	7,850
MLP	784 pixels $\rightarrow$ 256 ReLU features $\rightarrow$ 10 logits	203,530
CNN	Two convolution/pooling blocks $\rightarrow$ 64 features $\rightarrow$ 10 logits	105,866

2.3.1 Linear: a weighted pixel baseline

Each class receives a weighted sum of 784 pixels plus a bias, $z = Wx + b$. Without hidden features, the model cannot express nonlinear pixel interactions.

```
1 [B, 1, 28, 28] -> Flatten -> [B, 784]
2                 -> Linear(784, 10) -> [B, 10]
3 parameters = 784 x 10 + 10 = 7,850
```

2.3.2 MLP: nonlinear combinations of pixels

A hidden layer learns 256 features: $h = \max(0, W_1x + b_1)$, then $z = W_2h + b_2$. ReLU zeros negative activations, enabling nonlinear decision boundaries without explicitly representing spatial neighborhoods.

```
1 [B, 1, 28, 28] -> Flatten -> [B, 784]
2                 -> Linear(784, 256) -> ReLU -> [B, 256]
3                 -> Linear(256, 10) -> [B, 10]
4 parameters = (784 x 256 + 256) + (256 x 10 + 10)
5             = 203,530
```

2.3.3 Optional CNN: local patterns with shared filters

Two blocks learn local features with shared 3×3 filters, ReLU and 2×2 max pooling, followed by a dense classifier. Padding of one preserves spatial size during convolution. Each feature map contains one shared filter's responses. A 2×2 max-pooling window retains its largest response and halves each spatial dimension.

```
1 [B, 1, 28, 28]
2 -> Conv2d(1, 16, kernel=3, padding=1) -> ReLU
3 -> MaxPool2d(2) -> [B, 16, 14, 14]
4 -> Conv2d(16, 32, kernel=3, padding=1) -> ReLU
5 -> MaxPool2d(2) -> [B, 32, 7, 7]
6 -> Flatten -> [B, 1568]
7 -> Linear(1568, 64) -> ReLU -> Linear(64, 10) -> [B, 10]
```

2.4 Training and selection

Setting	Value
Random seed	42 for Python, NumPy and PyTorch; training shuffle also uses 42
Optimizer	Adam, learning rate 0.001
Budget	16 epochs, batch size 128; every epoch runs
Gradient clipping	Maximum total gradient norm 5.0
Selection	Highest validation macro-F1; earliest epoch wins an exact tie
Regularization	No augmentation, dropout, weight decay or learning-rate scheduler
Execution	CPU, four PyTorch threads, float32, deterministic algorithms enabled

Each batch clears gradients, computes cross-entropy, backpropagates, clips gradients and updates with Adam. Non-finite losses or gradients raise errors. Validation follows each epoch in evaluation mode without gradients.

Reported metrics use selected weights in `best.pt`; `checkpoint.pt` stores epoch-16 weights. Test results do not select checkpoints.

Comparison scope: architecture changes both capacity and image representation. This is not a capacity-matched ablation or hyperparameter search; equal epochs do not imply equal compute or optimization difficulty.

History records predictions while weights change. Checkpoint training metrics use fixed selected weights, so they may differ from that epoch's history.

2.5 Loss, metrics and timing

For logits z and target y , cross-entropy is $-z_y + \log \sum_k \exp(z_k)$. Batch loss is averaged; epoch loss weights batches by image count. Prediction uses $\text{argmax}(z)$ without post-processing.

- **Accuracy:** correct predictions divided by the number of images.
- **Macro-F1:** unweighted mean of ten class scores, $F1 = 2TP / (2TP + FP + FN)$; undefined scores are zero.
- **Training time:** the 16-epoch training and validation loop, including history writes and copies of best weights. It excludes dataset preparation, post-training fixed-weight metrics and final artifact generation.
- **Inference time:** the median of three complete batched passes after one warmup, divided by image count. It includes DataLoader iteration, the forward pass and argmax ; it excludes metric calculation.

Inference measures batched CPU throughput rather than single-request latency.

2.6 Recorded environment

Runs use Python 3.12.13, PyTorch 2.14.0, NumPy 2.5.3 and scikit-learn 1.9.1 on macOS 26.6.2 arm64, with four CPU threads and float32 computation.

2.7 Experiment design

- **Hypothesis:** nonlinear MLP features improve classification over a linear pixel classifier. The optional CNN tests local filters and spatial weight sharing.
- **Changed factors:** architecture, representation and parameter count.
- **Fixed factors:** input images, split, normalization, seed, optimizer, learning rate, batch size, epoch budget and evaluation.
- **Criterion:** higher validation macro-F1; report held-out accuracy and macro-F1 alongside parameter and runtime costs. Capacity also changes, so the experiment does not isolate nonlinearity or spatial structure as the sole cause.

2.8 Training and validation loop

The same epoch function receives an optimizer for training and no optimizer for validation. This controls both model mode and gradient recording. The following excerpt from `train.py` shows the update path.

Training/validation update path

```
1 training = optimizer is not None
2 model.train(training)
3 total_loss = 0.0
4 labels, predictions = [], []
5 with torch.set_grad_enabled(training):
6     for images, targets in loader:
7         images, targets = images.to(DEVICE), targets.to(DEVICE)
8         if training:
9             optimizer.zero_grad(set_to_none=True)
10            logits = model(images)
11            loss = nn.functional.cross_entropy(logits, targets)
12            if not torch.isfinite(loss):
13                raise FloatingPointError("non-finite loss")
14            if training:
15                loss.backward()
16                nn.utils.clip_grad_norm_(model.parameters(), CLIP_NORM, error_if_nonfinite=True)
17            optimizer.step()
```

Epoch loss weights each batch by its image count, including the smaller last batch. Accuracy and macro-F1 aggregate all predictions. Validation runs after every training epoch. The highest validation macro-F1 selects the saved weights; an exact tie retains the earlier epoch.



2.9 Reproduction

Run the following commands from the dl-co3133 repository root.

```
1 python3.12 -m venv .venv
2 .venv/bin/python -m pip install -r requirements.txt
3 .venv/bin/python Assignment1/download.py
4 .venv/bin/python Assignment1/data_loader.py
5 cd Assignment1
6 ../.venv/bin/python - <<'PY'
7 from train import set_seed, fit
8 from evaluate import evaluate
9 from models.linear import LinearClassifier
10 from models.mlp import MLP
11
12 set_seed(42)
13 linear_run = fit(LinearClassifier(), "Linear", 42)
14 set_seed(42)
15 mlp_run = fit(MLP(), "MLP", 42)
16 evaluate(LinearClassifier(), linear_run)
17 evaluate(MLP(), mlp_run)
18 PY
```

For the optional CNN, import CNN from `models.cnn` and use the same seed, `fit` and `evaluate` calls. Outputs are saved under `Assignment1/outputs/`. The selected `best.pt` contains weights, split indices and normalization; `checkpoint.pt` contains the final epoch weights. The EDA and results are recorded in `01_eda.ipynb` and `02_results.ipynb`.

3 Implementation results

Linear, MLP and the optional CNN use seed 42 and the same 16-epoch budget. All results use weights selected by validation macro-F1.

3.1 Validation comparison

Accuracy and macro-F1 use the same selected checkpoint. The accuracy gap is training minus validation accuracy in percentage points, measured with those fixed weights.

Model	Accuracy (0–1)	Macro-F1	Selected epoch	Epochs run	Train–val gap (pp)
Linear	0.8653	0.8658	12	16	0.2944
MLP	0.9068	0.9070	14	16	3.9111
CNN	0.9227	0.9233	14	16	2.9037

CNN has the highest validation macro-F1, **0.9233**, and is selected at epoch 14. Its validation accuracy is **92.27%**, compared with **90.68%** for MLP and **86.53%** for Linear. The MLP improves validation accuracy by 4.15 percentage points over the linear baseline.

At the selected weights, the training-minus-validation accuracy gap is 3.91 percentage points for MLP, 2.90 for CNN and 0.29 for Linear. The larger MLP gap suggests more overfitting; a small gap alone does not identify the strongest model.

Model	Parameters	Training (s)	Validation ms per image	Test ms per image
Linear	7850	5.9364	0.0023	0.0019
MLP	203530	11.8839	0.0028	0.0024
CNN	105866	151.3040	0.0360	0.0346

Linear is smallest at 7,850 parameters. CNN uses 105,866 parameters, about 52% of MLP's 203,530, but its measured test inference is about 14 times slower. Shared convolution weights reduce parameter count without eliminating repeated spatial computation.

Training and inference ran sequentially on CPU with four PyTorch threads. Training time includes the epoch loop, validation, history writes and in-memory copies of best weights. It excludes data preparation, post-training metrics and final checkpoint serialization.

Inference uses one warm-up pass followed by the median of three complete passes, including batching, forward computation and argmax, excluding metric calculation. These are batched throughput times per image, not single-image request latency. One training seed and one completed fit per model do not establish runtime variability.

3.2 Held-out test results

The official test set evaluates the validation-selected checkpoints.

Model	Loss	Accuracy (0–1)	Macro-F1	Inference (ms/image)
Linear	0.4438	0.8403	0.8408	0.0019
MLP	0.3505	0.8877	0.8878	0.0024
CNN	0.2638	0.9105	0.9108	0.0346

The validation-selected CNN reaches 91.05% test accuracy and 0.9108 macro-F1, below its validation accuracy by 1.22 percentage points. Linear reaches 84.03% and MLP 88.77%.

MLP improves over Linear by **4.74 percentage points**; CNN adds **2.28 points** over MLP. The validation and test rankings agree in this run. These are results for these implementations and one training seed, not a universal ranking of architecture families.

3.3 Learning curves

Dashed lines show training metrics during updates; solid lines show validation after each epoch. Vertical lines mark the selected checkpoints.

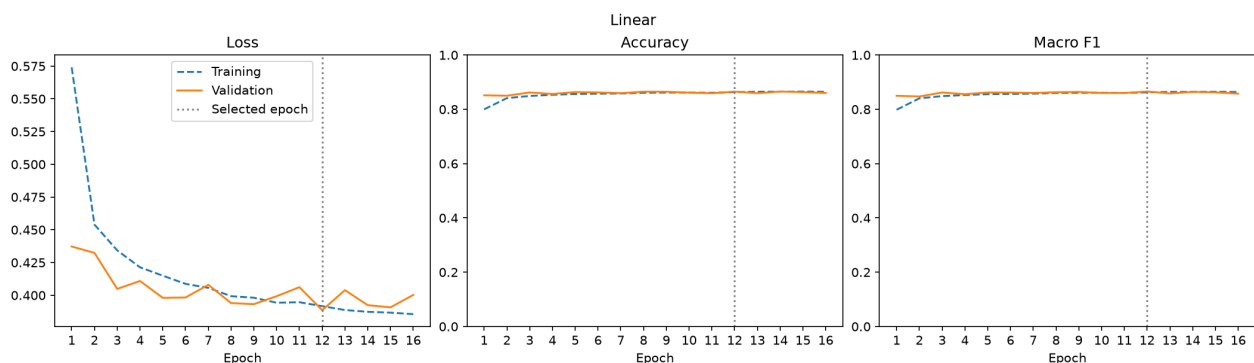


Figure 21: Linear — Learning curves.

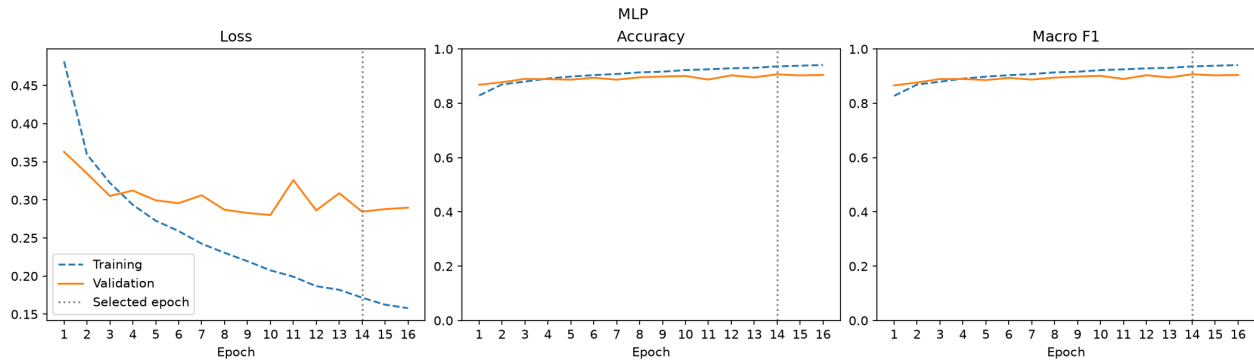


Figure 22: MLP — Learning curves.

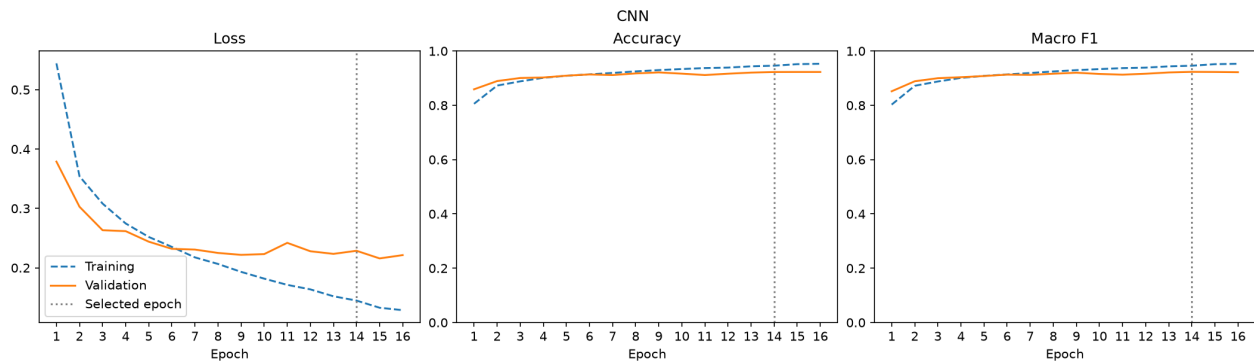


Figure 23: CNN — Learning curves.

All models complete 16 epochs. Linear selects epoch 12: validation macro-F1 is 0.8658 there and 0.8577 at epoch 16, while validation loss rises from 0.3886 to 0.4003. MLP selects epoch 14; its training loss continues falling while validation loss rises from 0.2845 to 0.2897 by epoch 16. The later weights are not necessarily the better generalizing weights.

CNN also selects epoch 14 even though epoch 16 has slightly lower validation loss and higher accuracy: macro-F1 falls from 0.9233 to 0.9219. Selected and final weights therefore differ for all three models. Dashed training curves are collected during updates and differ from the fixed-checkpoint training metrics above.

3.3.1 Validation images

The table counts correct predictions across all 6,000 validation images. The gallery shows the first 16 for CNN: green titles mark correct predictions, red titles mark errors. IDs refer to the original training file.

Model	Correct	Total	Accuracy (0–1)
Linear	5192	6000	0.8653
MLP	5441	6000	0.9068
CNN	5536	6000	0.9227



Figure 24: Validation images.

The displayed CNN page contains 15 correct predictions out of 16. Ankle boot #23942 is predicted as Sneaker with 86.1% confidence, illustrating a confident footwear error even on this mostly correct page. File order determines the examples shown; the table above uses all 6,000 predictions per model and gives CNN 5,536 correct (92.27%).

3.4 Test confusion matrices

Rows are true classes; columns are predictions. Cell counts show how often each decision occurs. Compare clothing confusions with the visual similarities observed in EDA.

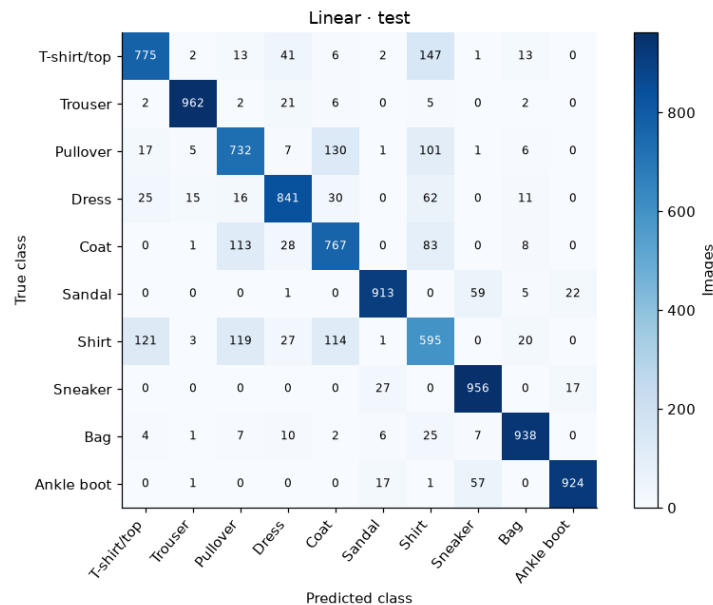


Figure 25: Linear — Test confusion matrices.

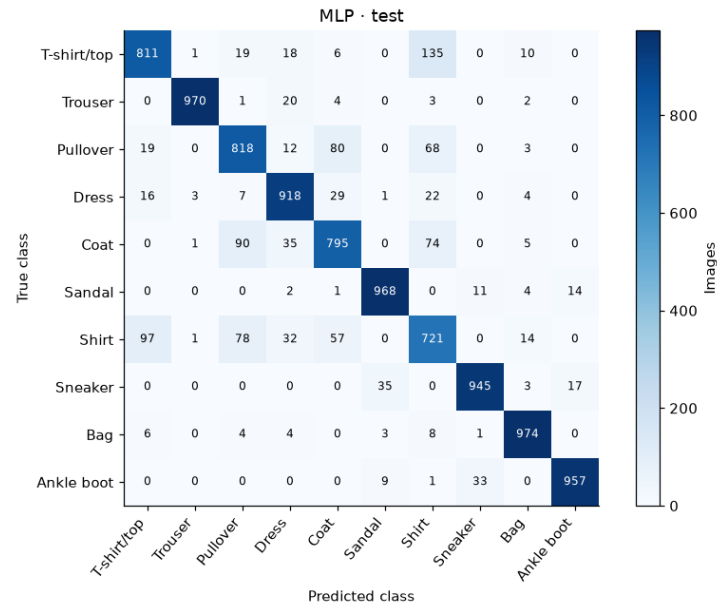


Figure 26: MLP — Test confusion matrices.

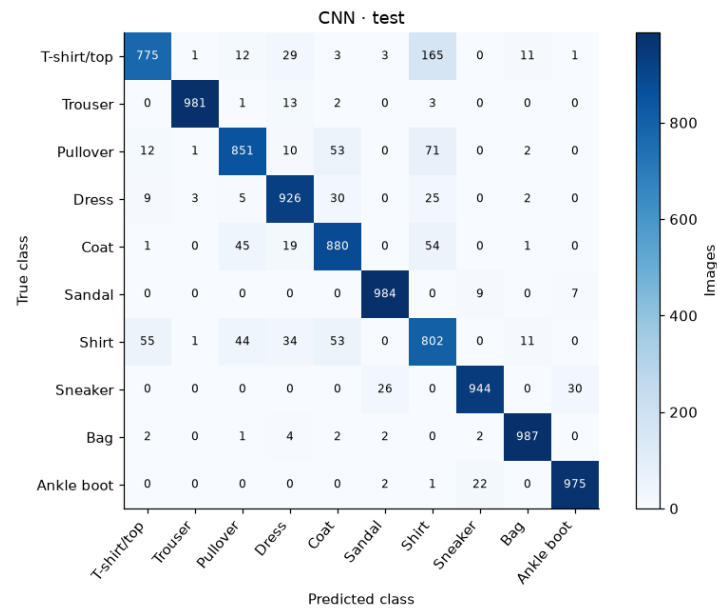


Figure 27: CNN — Test confusion matrices.

All three models confuse T-shirt/top, Shirt, Pullover and Coat, consistent with the EDA hypothesis about similar upper-body silhouettes. CNN reduces Pullover to Coat errors from Linear's 130 to 53, but makes 165 T-shirt/top to Shirt errors, versus 135 for MLP. Better overall performance does not improve every class pair.

Each true class contains 1,000 test images, so the displayed counts also convert directly to within-class percentages by dividing by ten. Shirt can attract many false positives even when true shirts are recognized correctly.

Table 1: Per-class test F1.

Class	Linear	MLP	CNN
T-shirt/top	0.797	0.832	0.836
Trouser	0.967	0.982	0.987
Pullover	0.731	0.811	0.869
Dress	0.851	0.900	0.910
Coat	0.746	0.806	0.870
Sandal	0.928	0.960	0.976
Shirt	0.589	0.710	0.756
Sneaker	0.919	0.950	0.955
Bag	0.937	0.965	0.980
Ankle boot	0.941	0.963	0.969

Shirt is the weakest class for every model: F1 ranges from **0.589** for Linear to **0.756** for CNN. CNN's Trouser F1 is **0.987** and Bag F1 is **0.980**, showing how much the difficulty varies despite equal class counts. CNN improves Shirt F1 over MLP's 0.710, but substantial overlap among upper-body garments remains. Macro-F1 retains that weak class's contribution instead of reporting only the overall success rate.

Model	True	Predicted	Count	Class error rate
Linear	T-shirt/top	Shirt	147	0.147
Linear	Pullover	Coat	130	0.130
Linear	Shirt	T-shirt/top	121	0.121
Linear	Shirt	Pullover	119	0.119
Linear	Shirt	Coat	114	0.114
MLP	T-shirt/top	Shirt	135	0.135
MLP	Shirt	T-shirt/top	97	0.097
MLP	Coat	Pullover	90	0.090
MLP	Pullover	Coat	80	0.080
MLP	Shirt	Pullover	78	0.078
CNN	T-shirt/top	Shirt	165	0.165
CNN	Pullover	Shirt	71	0.071
CNN	Shirt	T-shirt/top	55	0.055
CNN	Coat	Shirt	54	0.054
CNN	Pullover	Coat	53	0.053

CNN's five most frequent directional errors are T-shirt/top → Shirt (165; 16.5% of true T-shirts), Pullover → Shirt (71), Shirt → T-shirt/top (55), Coat → Shirt (54) and Pullover → Coat (53). Four of these five involve Shirt. The reverse Shirt → T-shirt/top direction is much less frequent than T-shirt/top → Shirt, so treating the pair as a symmetric error would hide useful information.

3.5 Inspect predictions

For each model, show four correct predictions in test-file order, four lowest-confidence correct predictions, and four incorrect predictions in test-file order. Confidence is the largest softmax probability; it is not a calibrated guarantee. IDs refer to the official test image file.

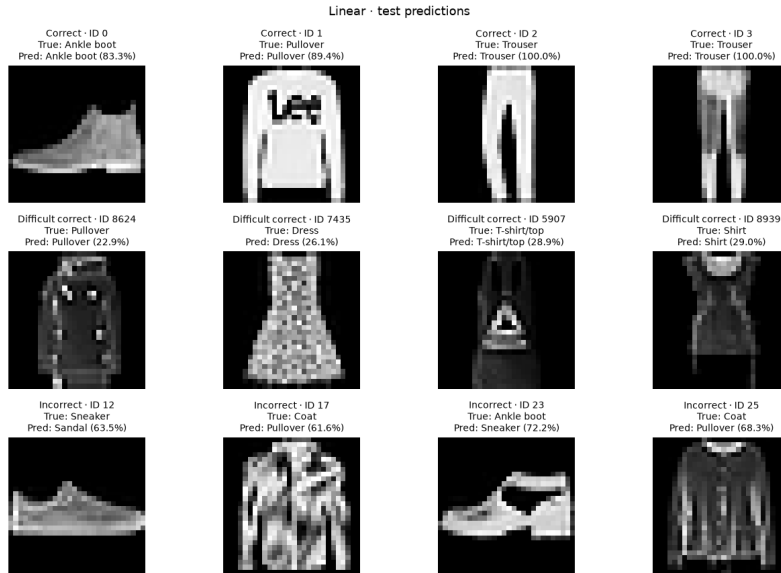


Figure 28: Linear — Inspect predictions.

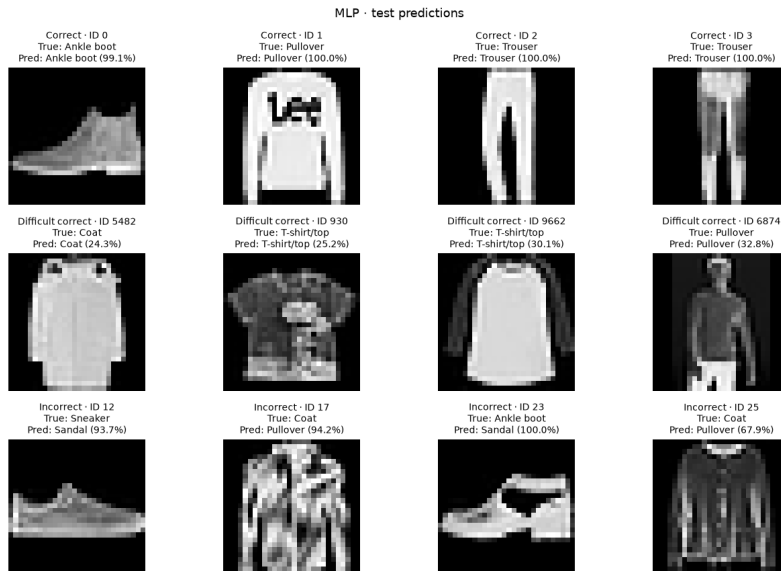


Figure 29: MLP — Inspect predictions.

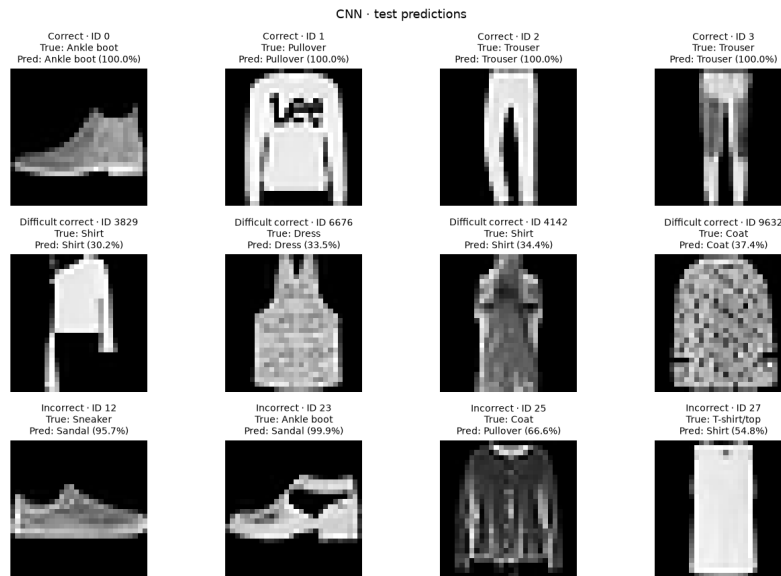


Figure 30: CNN — Inspect predictions.

CNN correctly recognizes the first boot, printed pullover and two trousers, whose broad silhouettes are visible. Its low-confidence correct cases include Shirt #3829 at 30.2% and Dress #6676 at 33.5%; the pictured garments have unusual or indistinct outlines. They show that a correct class can win with low confidence.

CNN misclassifies Sneaker #12 as Sandal at 95.7% confidence and Ankle boot #23 as Sandal at 99.9%. The latter image has a heel and large dark opening that resemble a sandal despite its supplied label. CNN also labels Coat #25 as Pullover and T-shirt #27 as Shirt. These examples illustrate visual ambiguity without establishing a labeling error.

Linear recognizes the first boot, pullover and trousers but predicts Coats #17 and #25 as Pullovers. Correct Pullover #8624 has 22.9% confidence. MLP correctly labels Coat #5482 at 24.3% confidence, yet misclassifies Sneaker #12 as Sandal at 93.7%. High softmax confidence does not guarantee correctness; the first four mistakes form a deterministic inspection sample.

3.6 Discussion

The 256-unit MLP adds nonlinear features to the flattened 784-pixel representation and improves test accuracy from **84.03% to 88.77%** over Linear. Both lose explicit two-dimensional neighborhood structure, but the MLP can combine pixels nonlinearly. The models also differ in parameter count and computation, so the comparison does not isolate nonlinearity as the sole cause of the gain.

CNN is the validation-selected candidate. Its local 3×3 filters and pooling exploit spatial structure, reaching **91.05% test accuracy** and **0.9108 macro-F1**. Against MLP it gains 2.28 accuracy points with roughly half as many parameters, at about 14 times the measured batched CPU inference time per image. CNN is the stronger accuracy choice here; MLP offers a useful inference-cost trade-off. The sequential fit takes 151.30 s for CNN versus 11.88 s for MLP over the same 16-epoch budget; inference and training costs should be assessed separately.

The persistent limitation is upper-body clothing ambiguity. Shirt remains CNN's weakest class, and **165 true T-shirts are predicted as shirts**. The inspected footwear mistakes also show that high softmax confidence does not guarantee a correct label. This links the result analysis back to EDA's overlapping silhouettes while keeping descriptive image similarity separate from measured classification errors.

Repeat the shared setup with additional seeds, then use training and validation data to assess modest image shifts or weight decay. The present comparison uses one split, one training seed and a fixed budget. These runs do not establish training variability, statistical significance or each architecture's best attainable performance. Small grayscale catalog images also limit transfer to real-world photographs.